# Paper 007: Prediction, execution, and learning evidence

## Study guide: Paper 007

**AuthorityEffect:** None. This guide prepares the quiz; the instructor key is a separate file. The preface uses fictional examiners. The mathematics and evidence come from the paper, the local notebook, and explicitly declared examples.

You need integer arithmetic, fractions, and a way to read a directed edge. Study the complete calculation, cover a step and reconstruct it, then predict a changed instance before running code. Keep the first response and any later correction. The eight levels below describe an instructional sequence, not an empirically calibrated theory of learning.

### Levels 0–1: One carrier and a complete calculation

A graph carrier declares vertices and edges. This triangle has ordered edges $ab, bc, ca$, rational coefficients, and no filled face. The arrow from $s$ to $t$ means that the difference is $f_t - f_s$. A **claim table** $h$ assigns one rational number to each directed edge; a **potential** $f$ assigns one to each vertex. The residual is $r = h - df$.

| Edge | Claim $h$ | Source height | Target height | Difference $df$ | Residual $r$ |
|---|---|---|---|---|---|
| $ab$ | $1/3$ | $0$ | $1/3$ | $1/3$ | $0$ |
| $bc$ | $1/3$ | $1/3$ | $2/3$ | $1/3$ | $0$ |
| $ca$ | $1/3$ | $2/3$ | $0$ | $-2/3$ | $1$ |
| Signed sum | $1$ | — | — | $0$ | $1$ |

The difference moves the discrepancy onto one edge. It cannot make all three residual coordinates zero because every potential difference sums to zero around this cycle, while the claims sum to one. Here “sum” means the signed sum in

the declared order. It is not a norm, a weighted optimization result, or a native solver certificate.

Use `Fraction(1, 3)` or `Fraction("1/3")` to specify 1/3 directly. In `Fraction(1/3)`, Python first performs floating-point division. The resulting fraction exactly represents that float, which can differ from 1/3. Exact arithmetic does not repair an input that already represents a different number. The worksheet below does not include a general input validator.

**Try before checking:** add the same rational $k$ to every height. On every edge,

$$(f_t + k) - (f_s + k) = f_t - f_s.$$

Therefore the differences and residual stay fixed. Negating the claims while keeping the potential gives $-h - df$, not $-(h - df)$. Write out both expressions to locate the sign difference.

## Level 2: A proof, then a changed instance

For any directed closed walk $v_0, \ldots, v_m = v_0$,

$$\sum_{i=0}^{m-1} (df)_{v_i v_{i+1}} = \sum_{i=0}^{m-1} (f_{v_{i+1}} - f_{v_i}) = 0.$$

Each height occurrence cancels with the adjacent occurrence, including at the closing edge. Repeated vertices do not invalidate this telescoping calculation. It follows algebraically that $C(h - df) = C(h)$ on the walk. This is a general identity under the stated definitions; checking a few potentials is a finite illustration of it.

For this particular triangle, a zero cycle sum also lets us construct a potential: set $f_a = 0$, $f_b = h_{ab}$, $f_c = h_{ab} + h_{bc}$. The closing difference is then $-h_{ab} - h_{bc}$, equal to $h_{ca}$ exactly when the sum is zero. Do not silently extend this single-cycle criterion to arbitrary graphs without checking every relevant cycle.

A path has no closing constraint. With path claims $(1, 2)$, heights $(0, 1, 3)$ realize both edges. Closing it with claim 1 creates residual $(0, 0, 4)$. Changing the closing claim to $-3$ changes the claim table and makes it exact. Attaching a filled cell changes the carrier and adds further semantics; the drawing alone performs neither operation.

**Completion practice:** cover the closing claim, derive its value, and label explicitly whether you changed a claim, a potential, or the carrier.

## Level 3: Make the checker discriminate

A useful control is a plausible implementation fault with a known consequence. Reversing the difference gives $\widetilde{df} = (-1/3, -1/3, 2/3)$. Its signed sum is still zero and the wrong residual $(2/3, 2/3, -1/3)$ still sums to one. A cycle-sum

checker alone cannot discriminate the sign convention. The independently worked coordinate table can.

Different controls check different claims. A parser needs an invalid-syntax example. A digest check needs changed content bound to an unchanged digest. A finite LP certificate needs computed objectives **and** primal/dual feasibility. For minimize nonnegative $x$ with $x \geq 1$, and dual maximize $y$ with $0 \leq y \leq 1$, the pair $x = y = 0$ has equal objectives but an infeasible primal. The pair $x = y = 1$ satisfies both feasibility and equality.

The legacy status-grading example scans for the substring `assert`. A comment or string containing that word can enter its denominator. It reads supplied statuses and does not execute code. Even a more careful assertion count would need a declared scoring interpretation; assertions are not participant answers.

**Practice:** say which property your control can falsify, why it should fail, and which faults it could still miss.

## Level 4: Keep source identity and execution separate

| Evidence | Supports | Leaves open |
|---|---|---|
| Stored outputs and execution counts | The deposited JSON contains those fields | Whether a cell executed now, or a person predicted it |
| Fresh isolated notebook replay | These cells execute in the declared process/backend; matching outputs reproduce this reference run | General theorem, learner mastery, full repository integration |
| Hash and file-census verification | Listed bytes and archive contents agree | Mathematical truth, pedagogical quality |
| Historical source-bound receipt | The recorded outcome for its stated inputs and environment | Outcome after source or API changes |
| Source inspection | What the inspected version declares or implements | Fresh execution and uninspected dependencies |

From the package root, install the declared development dependencies, then replay:

```
python -m pip install -e '.[dev]'
python tools/replay_companion_notebook.py
python -m pytest code/tests -q -ra
```

The default replay executes ordinary Python cells in a fresh CPython process and compares sequential cell counters and normalized outputs without writing executed results into the deposited notebook. The counters are not Jupyter kernel history. An optional `--backend jupyter --kernel NAME` replay requires a registered kernel. That backend could not run on the review host because sockets were unavailable; CPython success does not establish Jupyter rendering or IPython behavior. It verifies that the file did not change during replay. The deposit's stored outputs may be updated intentionally when authoring a new candidate, followed by a new receipt and manifest.

The source inventory pins inspected sources and the 20 route notebooks at audit commit `52cb5457f6c485ba548cfd292f6012a941ed22cf`. If a cited file moves, obtain the original commit's bytes and check identity, or record that they are unavailable. A different current implementation needs its own labeled run. A same-name file is not a historical substitute.

The saved 20/20 route receipt has old paths; a route notebook also uses a retired norm API. The local fraction lesson avoids that interface. Its success does not refresh the external route.

## Level 5: Read every formal hypothesis

The list bound concerns an integer list and a natural number $n$:

$$\operatorname{mass}(h) = \sum_{x \in h} |x|, \quad |h| \le n, \quad |x| \le n - 1 \quad \Longrightarrow \quad \operatorname{mass}(h) \le S(n).$$

Here $S(n) = 0$ for $n < 2$, otherwise $n(n-1)$; subtraction on natural numbers truncates at zero. At $n = 3$, the maximum follows by counting at most three entries of magnitude at most two. A magnitude cap without a length bound cannot supply this result. These are declared finite mathematical conditions, not assumptions validated for human minds.

The unary nesting lemma preserves supplied symbol/operator declaration predicates when the nested operator is allowed and the starting term satisfies the predicates. It permits arbitrary finite nesting depth; it does not impose a maximum depth or a search-time bound.

The named report has 21 selected declarations. The full package-local report has 64, including generated/private declarations, and excludes imported library declarations. Distinguish permitted axioms from observed dependencies. Reading a saved report is not rerunning Lean.

**Practice:** for each theorem state the carrier, assumptions, conclusion, and a tempting interpretation it does not support.

## Level 6: Record what a learner did

A code result and a learner result have different units of observation. A learner assessment needs the person's first response to a task under a stated exposure

condition and an explicit scoring rule. A proposed minimal record is:

| Field | Purpose |
|---|---|
| Participant pseudonym, task/version, source hashes | Identify the learner observation and task bytes |
| Baseline task response and assigned condition | Record prerequisites and comparison |
| First prediction, explanation, submission/exposure order | Distinguish prediction from answer reproduction |
| Feedback shown and corrected response | Retain learning from feedback without replacing the first response |
| Novel transfer task, response, independent score | Assess a prespecified skill on an unfamiliar instance |
| Delayed response or explicit missing value/reason | Preserve the denominator and missingness |

For a proposed study, define the compared lesson formats, assignment procedure, assessment tasks, scoring, exclusions, timing, missing-response handling, and analysis before inspecting results. Use tasks whose answers are not supplied by the lesson, and retain the number assigned and assessed in each condition. A corrected response after seeing the key is not a first prediction. A missing response is not automatically a zero or a pass; use the prespecified analysis and disclose missingness.

These requirements outline a future investigation. This package has no participant corpus, no measured attention half-life, and no estimate of transfer effect.

## Level 7: Write the strongest supported sentence

Build a lesson with a declared example, visible derivation, first prediction, changed-input probe, and boundary. State what was actually checked: "The declared rational triangle reproduces the displayed residual and rejects the deliberate wrong-sign implementation by its coordinates" is assessable. "All students transfer cycle reasoning" needs human observations and a study design beyond the cells.

For publication, distinguish standalone tests, isolated source-module checks, a full repository run, formal-kernel replay, and historical records. A local release archive is a publication candidate; a verified archive and a proposed grading rubric do not create a DOI or certify a learner.

## Exact worksheet

Save the following as a scratch Python file, or run it in a fresh cell. Predict the rows on paper first. It imports only the standard library and checks arithmetic

and a deliberate mutant. It contains reference answers; running it is not an assessment of the reader.

```
from fractions import Fraction as Q

edges = (("a", "b"), ("b", "c"), ("c", "a"))
h = (Q(1, 3), Q(1, 3), Q(1, 3))
f = {"a": Q(0), "b": Q(1, 3), "c": Q(2, 3)}
df = tuple(f[t] - f[s] for s, t in edges)
r = tuple(claim - delta for claim, delta in zip(h, df))
assert df == (Q(1, 3), Q(1, 3), Q(-2, 3))
assert r == (Q(0), Q(0), Q(1))
assert sum(df, Q(0)) == 0 and sum(r, Q(0)) == 1

shifted = {v: height + Q(7, 5) for v, height in f.items()}
assert tuple(shifted[t] - shifted[s] for s, t in edges) == df
negated = tuple(-claim - delta for claim, delta in zip(h, df))
assert negated == (Q(-2, 3), Q(-2, 3), Q(1, 3))

wrong_df = tuple(f[s] - f[t] for s, t in edges)
wrong_r = tuple(claim - delta for claim, delta in zip(h, wrong_df))
assert sum(wrong_df, Q(0)) == 0 and sum(wrong_r, Q(0)) == 1
assert wrong_df != df
assert wrong_r == (Q(2, 3), Q(2, 3), Q(-1, 3))

completed = (h[0], h[1], -h[0] - h[1])
assert completed == df
assert Q(1, 3) == Q("1/3") and Q(1 / 3) != Q(1, 3)
assert sum(abs(x) for x in (2, -2, 1)) == 5 <= 3 * (3 - 1)
print("triangle, changed-input probes, and wrong-sign control pass")
```

After feedback, create a different cycle and give its orientation, claims, potential, predicted residual, and evidence sentence. Revisit levels 0–2 for arithmetic/scope errors, levels 3–5 for checker/source/formal errors, and levels 6–7 for assessment/publication errors. Retain the first and revised attempts. Read the instructor key only after completing the corresponding quiz.